■ Design Autocomplete / Typeahead

System Design Interview | Search & Aggregation

algomaster.io | Deep Dive Notes

■ Key Requirements

Functional	Return top-K suggestions for prefix in real-time; Suggestions ranked by frequency/relevance; Personalized and global suggestions
Non-Functional	Response <100ms (p99); Handle 10K queries/sec; Real-time updates from new searches; Multi-language support
Scale	Google: 8.5B searches/day; Each keystroke = API call; Prefix tree (Trie) is the core data structure

■ Trie-Based Design

• Trie: prefix tree where each node = character; path from root = string prefix; leaf or marked node = complete word

• Each node stores: character, children map, is_end_of_word, frequency (for ranking), top-K cache

• Top-K at each node: cache top-10 suggestions for that prefix; avoids DFS traversal on every query → O(1) lookup after build

• Memory optimization: compress common prefixes (Radix Tree / Patricia Trie); reduces nodes by ~50%

■ Architecture Deep Dive

• **Query Path:** Client → API Gateway → Autocomplete Service → look up prefix in Trie (in-memory) → return top-K; sub-millisecond with in-memory trie

• **Update Pipeline:** Search logs → Kafka → batch aggregation (hourly/daily) → recompute Trie → replace old Trie atomically

• **Distributed Trie:** Partition trie by first 2 chars (26×26 = 676 partitions); each server owns subset; Zookeeper maps prefix → server

• **Caching:** Most queries are popular (power law); Redis cache for top-10K prefixes; cache hit rate ~80% with 1M cache entries

■ Data Storage

• Serialized Trie: store in memory (primary) + serialize to Redis/S3 for persistence and replication

• Alternative: store (prefix, top_k_list) pairs in Redis hash; update on search log aggregation

• Full text: Elasticsearch with edge n-gram tokenizer for flexible prefix matching; slower but handles typos with fuzzy search

• Frequency DB: Cassandra stores (query, frequency) pairs; batch job reads these to rebuild Trie

■ Personalization & Ranking

• Global ranking: query frequency across all users (last 7-30 days); recomputed daily via batch job

• Personalization: user's recent searches + location bias merged with global at query time (weighted blend)

• Contextual: if user is on travel page, boost travel-related suggestions for ambiguous queries

• Ranking formula: score = α×global_freq + β×personal_freq + γ×recency + δ×context_boost

■■ Storage Trade-offs

In-Memory Trie	Elasticsearch
✓ Sub-millisecond latency	✓ Fuzzy/typo matching
✓ Full control over ranking	✓ Easy to update
✓ Memory: ~10GB for 1B queries	✓ 50-100ms latency
✓ No typo tolerance	✓ More operational complexity

■ Scale Optimizations

• Client-side debouncing: wait 100-200ms after keystroke before API call; reduces requests by 60-70%

• Client-side caching: cache last 10 queries in browser/app; instant suggestions for backspace

• Trie snapshot: rebuild trie every hour from frequency DB; swap atomically (blue-green); no downtime

• Filter offensive/sensitive queries from suggestions; separate filtering service or blocklist in trie